Full task list

A. Dataset & Retrieval

1. Load **MSMARCO-XI Hindi validation subset** and subsample ~2–5k rows.
2. Build **fixed-size chunking with overlap**.
3. Build **semantic chunking**.
4. Build **metadata-aware/hierarchical chunking**.
5. Generate embeddings and build **FAISS dense index**.
6. Build **BM25 sparse index**.
7. Implement **RRF fusion** to combine dense + sparse retrieval.
8. Evaluate retrieval using `is_selected` ground truth and **Precision@k** for all chunking strategies.
9. Create a retrieval comparison between the three chunking strategies.
10. Test retrieval on **Hindi, English, and Hindi-English code-mixed queries**.

B. Voice & Generation

11. Integrate **Sarvam Hindi STT**.
12. Add **STT timeout and retry handling**.
13. Extract **STT confidence score**.
14. Add **"Please repeat" fallback** for low-confidence transcription.
15. Build generation-call wrapper with a prompt containing retrieved Hindi chunks.
16. Implement **citation tagging** so the generated answer identifies the source chunks used.

C. Guardrails & Anti-Hallucination

17. Build **off-topic query classifier**.
18. Build **unsafe-input filter**.
19. Build **multi-signal grounding checker**:

- Embedding similarity
- LLM self-critique
- Citation validity

20. Build **"I don't know" pathway** when evidence is insufficient.
21. Build a small **grounding/hallucination evaluation set** containing:

- Answerable questions
- Unanswerable questions
- Partially supported questions

22. Evaluate whether the system correctly **answers vs. refuses** unsupported questions.

D. Pipeline & Reliability

23. Build `PipelineRunner` connecting:

1. STT
2. ↓

3. Input Guardrail
4. ↓
5. Retrieval
6. ↓
7. Generation
8. ↓
9. Grounding Check
10. ↓

Answer / "I don't know"

24. Add **structured input/output** across every stage.
25. Add **error handling** across the pipeline.
26. Add **per-stage timeout/failure handling**.

E. Performance

27. Build latency benchmarking for **50–100 test queries**.
28. Record timestamps for every pipeline stage.
29. Calculate **P50, P70 and P100 latency**.
30. Build **per-stage latency breakdown**.
31. Build a **live latency dashboard** showing STT, guardrail, retrieval, generation, grounding and total latency.

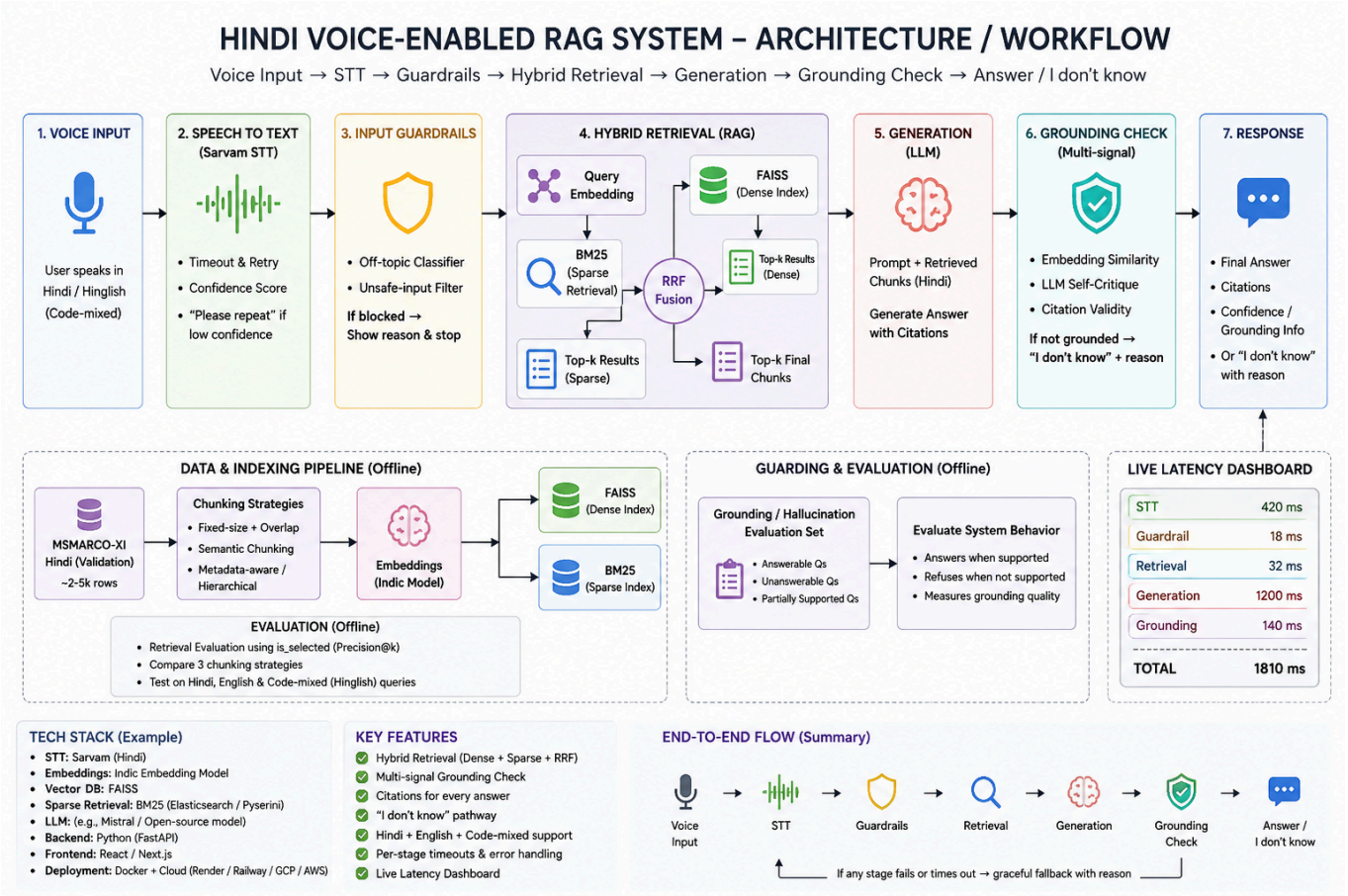
F. Frontend & Deployment

32. Build frontend with:

- Microphone input
- Recording/loading state
- Answer display
- Citations
- Guardrail decisions
- `"I don't know"` reason
- Grounding/confidence information
- Live latency

33. Connect frontend to the complete backend pipeline.
34. Deploy the **fully working system** with a live accessible link.

Pipeline Workflow (how data flows through the system)



Work Split

Aditya Vikram — Voice + Generation track (fully independent)

Independent work

13. Integrate Sarvam Hindi STT.
14. Implement STT timeout/retry.
15. Extract STT confidence.
16. Implement `"Please repeat"` fallback.
17. Build generation wrapper.
18. Design prompt for retrieved Hindi context.
19. Implement citation tagging.

Interface:

transcribe(audio)
→ {text, confidence}

generate(query, chunks)
→ {answer, citations}

Small additional responsibility

20. Build **voice-side test cases**:
- Hindi
- English
- Hindi-English mixed speech
- Low-confidence speech
- Empty/unclear speech

He doesn't need Shraddha's retrieval system.

He can simply use:

MOCK RETRIEVED CHUNKS
↓
generate()

So Vikram can finish his entire module without waiting for RAG.

Akshay — Harness + Guardrails + Frontend track (fully independent)

Independent work

21. Off-topic query classifier.
22. Unsafe-input filter.
23. Multi-signal grounding checker:
- Embedding similarity
- LLM self-critique
- Citation validity

24. `"I don't know"` pathway.

25. Build grounding/hallucination evaluation set:
- Answerable
- Unanswerable
- Partially supported

26. Implement `check_grounding()`.

Interface:

check_grounding(answer, citations, chunks)
→ {grounded, reason}

Small additional responsibility

27. Build `PipelineRunner` using mocks.

28. Add structured I/O and error handling.

29. Add per-stage latency logging.
30. Build initial frontend shell.

He can simulate everything:

MOCK STT
↓
Guardrail
↓
MOCK RETRIEVAL
↓
MOCK GENERATION
↓
Grounding
↓
Answer / I don't know

So Akshay also doesn't wait for either of you.

Shraddha — Retrieval track (fully independent)

Independent work

1. Load MSMARCO-XI Hindi validation subset.
2. Subsample ~2–5k rows.
3. Implement fixed-size + overlap chunking.
4. Implement semantic chunking.
5. Implement metadata-aware/hierarchical chunking.
6. Generate embeddings.
7. Build FAISS dense index.
8. Build BM25 sparse index.
9. Implement RRF fusion.
10. Implement:

retrieve(query, k)
→ [{chunk_text, score, source_id}]

Small additional responsibility

11. Evaluate **Hindi + English + Hindi-English code-mixed queries**.
12. Compare the 3 chunking strategies using `is_selected` and Precision@k.

She can work 100% independently

Use a hardcoded test query and don't wait for STT.

Typed Hindi query

↓
retrieve()
↓
Top-k chunks

Integration (only step that needs everyone, and it's fast because interfaces already match)

AFTER ALL 3 FINISH THEIR MODULES

Then you integrate.

Step 1 — Replace mocks

Akshay's:

- *MOCK STT*
- *MOCK RETRIEVAL*

MOCK GENERATION

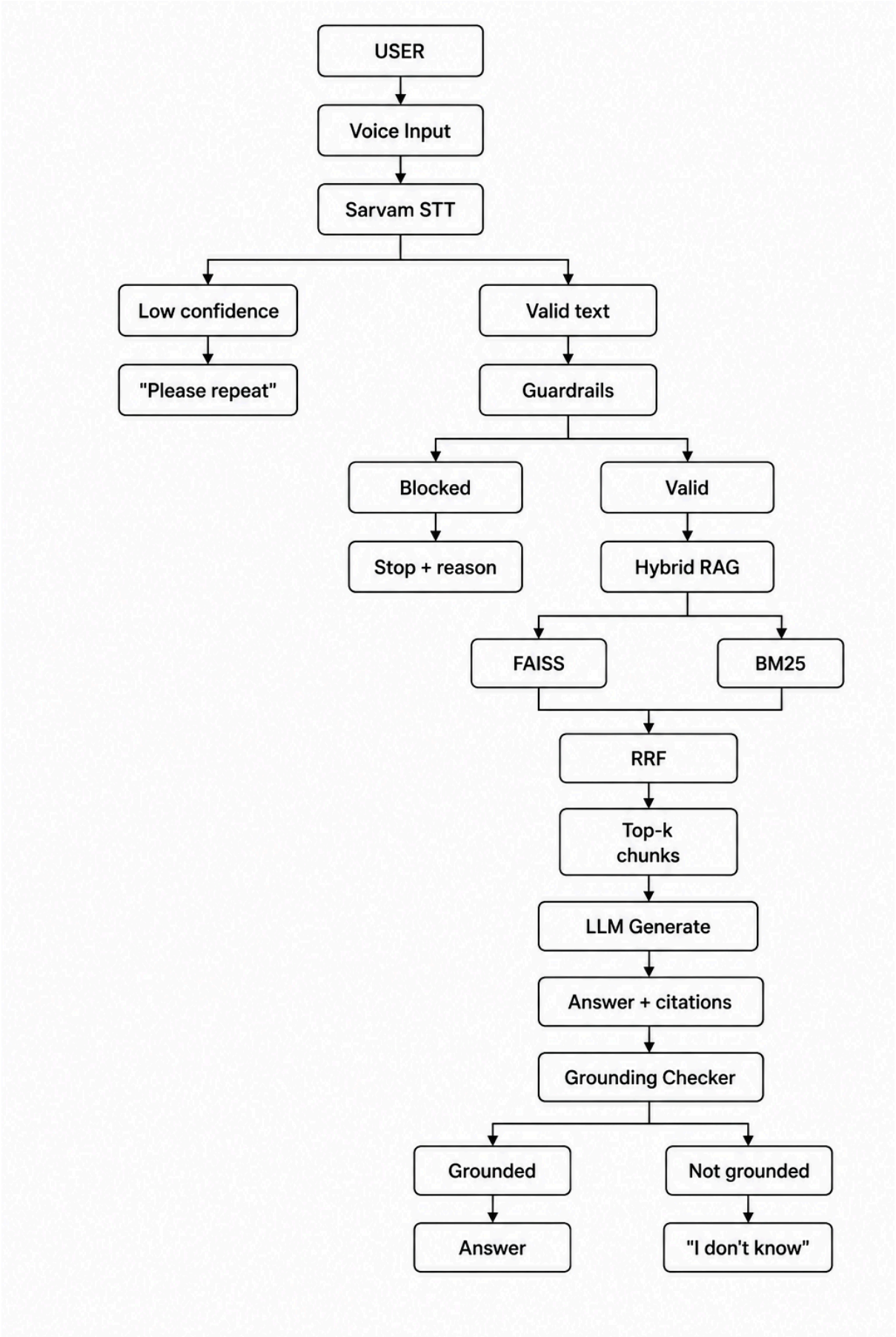
becomes:

- *Vikram's STT*
- *Shraddha's retrieve()*

Vikram's generate()

Then connect Akshay's grounding checker.

Shared Integration Pipeline


USER
Voice Input
Sarvam STT
Low confidence
"Please repeat"
Valid text
Guardrails
Blocked
Stop + reason
Valid
Hybrid RAG
FAISS
BM25
RRF
Top-k chunks
LLM Generate
Answer + citations
Grounding Checker
Grounded
Answer
Not grounded
"I don't know"

Then shared performance work

*After integration, **all 3 work together** on:*

Latency

- *50–100 test queries*
- *STT latency*
- *Guardrail latency*
- *Retrieval latency*
- *Generation latency*
- *Grounding latency*
- *Total latency*
- *P50 / P70 / P100*

Live dashboard

- *STT 420 ms*
- *Guardrail 18 ms*
- *Retrieval 32 ms*
- *Generation 1200 ms*
- *Grounding 140 ms*
- ——————————————

TOTAL 1810 ms

Final testing

Test:

- *Normal Hindi*
- *English*
- *Hinglish/code-mixed*
- *Off-topic*
- *Unsafe*
- *Unsupported question*
- *Low-confidence speech*
- *Correctly grounded answer*
- *Incorrect/unsupported answer*